Cutting Copilot CLI token billing: synthesis for the work group

Written 13 Oct 2026 from the project plan (`PLAN.md`) and the 10 Oct management report (data as of 10 Oct). Measured on one developer's workstation: check your own numbers (section 6).

1. In short

- **About 80% of the bill is context carrying, not output.** Before every step the assistant re-reads (cache read, 51%) and re-stores (cache write, 29%) the whole conversation. Written answers are 18% and new input 2%.
- **Three behaviours create most of the waste:** a very large context window, long autopilot runs, and resuming idle sessions after the 5-minute prompt cache has expired.
- **Done on 9 Oct:** smaller default context window and a per-session spending cap. A local usage logbook with a weekly report is also in place.
- **Goal: −50% at the same quality; expected: −30 to −40%.** Nothing is proven yet: the first before/after comparison is due around 16 Oct.
- **Surprise:** auto-approving safe commands was expected to save 2–4% and measured about 0%.
- **Method:** measure first, change one lever at a time, check quality indicators weekly.

Unit: AIU is the CLI's billing unit (`total_nano_aiu / 1e9`; 1 AI credit = $0.01).

2. Issues found (what drives the bill)

Dataset: ≈140k AIU in 203 sessions with cost records (24 Jul – 10 Oct; a lower bound).

#	Issue	Evidence
1	Context carrying dominates	Every step re-reads the whole history (≈180k tokens on average): cache read + write ≈80% of AIU
2	Very large window by default	199 of 200 sessions with a recorded tier ran `long_context` (936k window, compaction only at ≈750k). 81.5% of AIU comes from prompts ≥100k tokens, 21.6% from prompts >400k
3	Idle gaps expire the 5-minute prompt cache	Rebuilding costs ≈12× a cached read (250 vs 20 AIU per 1M tokens). Cold rebuilds are ≈19–20% of AIU: ≈12% from users resuming idle sessions (≈3 h idle on average; ≈64 AIU for the first step vs ≈11 for a fresh session) and ≈7% from agent-side waits (approvals, `ask_user`, `sleep` / `read_bash` polls, sub-agents)
4	Autopilot	91 sessions = 73% of AIU; each costs ≈3.3× a hands-on session
5	A few long sessions decide the bill	31 sessions (15%) = 71% of AIU; 12 sessions = 50%
6	What gets carried (estimates)	Fixed prompt ≈15% of spend (≈42k tokens before your first message: system ≈13k, 29 tool schemas ≈13k, 44 skills ≈2.5k, instructions); assistant messages and tool arguments ≈12%; file views ≈9%; bash search/read ≈3%; builds ≈3%; web ≈1.5%. Each extra MCP tool adds ≈450 tokens to every call
7	Helper agents on a pricey model	Sub-agents ≈10–11% of AIU; `gpt-5.4` helpers average 4.1 AIU/call vs 0.8 for `gpt-5.4-mini`
8	Little batching	Only 13.8% of tool-calling messages issue more than one call; at most 9.5% of calls could be merged
9	Approval waits	7,436 prompts, ≈110 h cumulative wait (includes time users were away); top triggers: compound `cd … &&` commands, grep, echo, git, cat, find, ls

Measured and **not** big drivers: reasoning effort `max` (minor lever) and large tool outputs (the CLI caps results at ≈8k tokens; results of 3–8k tokens carry only 2.8% of AIU).

3. Optimisations and expected saving

Lever	What	Est. saving	Status
Default context window	`contextTier: default`: history is compacted at ≈160k tokens instead of ≈750k. The large window stays one flag away	15–20%	**Done 9 Oct**; all 14 new sessions since then use it
Spending cap	`--max-ai-credits 1000` in the `copilot` alias: a forced decision point	not quantified (aims at the 30 sessions above 1,000 AIU = 72% of spend)	Done 9 Oct, see open points
Fresh session + hand-over note	Start fresh with an automatic ≤1,500-token summary instead of resuming a stale session	up to 8–10%	To build
Cheaper helper models	explore/task sub-agents on a cheaper model via `/subagents`; review and research agents unchanged	3–4%	From ≈16 Oct
Working habits	Standing instructions: batch independent calls (3–5%), poll slow jobs at ≤240 s (≈1%), hand noisy build loops to a helper (1–2%)	5–8%	From ≈16 Oct
Search past work	On-demand history/code lookup instead of re-reading	1–3%	Planned, only if a replay test supports it
Auto-approve read-only commands	Permission hook	≈0% (measured)	Observation mode only

Upper-bound estimates that overlap: do not add them up.

Tooling built (local to one workstation): a PostgreSQL logbook of all sessions with secrets masked before storage, a weekly report, a permission hook in observation mode (≈10–14 ms per decision) and 319 automated tests. The largest levers are settings and habits; the custom tooling mainly provides measurement and the hand-over note.

4. Dead ends and lessons

- **Auto-approving safe commands.** Replay of 4,536 real approval requests: the hook would answer 9% of them and none of the 43 that people refused or cancelled. But those were the quick ones: 0.1% of the waiting time and none of the 118 waits of ≥5 minutes (the ones that rebuild the cache). Not counted on.
- **Trimming tool results:** nothing exceeds ≈8k tokens; ≤2% gain with a quality risk.
- **Web/doc cache:** web is ≈1.5% of spend.
- **LLM gateway or proxy:** needs TLS interception or bring-your-own-key.
- **Local model (Ollama) instead of Copilot:** CPU-only, ≈22 tok/s prompt and ≈6.5 tok/s generation, so the 42k-token fixed prompt alone would take ≈30 minutes to ingest. Fine for small one-shot background jobs only (<1% gain).
- **Registering a 10-tool code-graph MCP server:** +3–4k tokens on every call; kept as a skill/CLI instead.
- **Parked (≤2%):** effort `max` → `high`; disabling 22 never-used skills (≈0.4%).

5. What you can do today

1. **Default window, large one on request.** `~/.copilot/settings.json`: `"contextTier": "default"`; per run: `copilot --context long_context`.
2. **Set a spending cap.** `copilot --max-ai-credits 1000` (minimum 30), or `/limits set max-ai-credits <n>` in a session (survives `/clear` and `/new`). It cannot be persisted in the config (no key, no env var), so use a shell alias and check `/limits` at session start. At the cap the CLI pauses for a decision; one call can overshoot.
3. **Bound autopilot runs** with a lower cap, e.g. `copilot --autopilot --max-ai-credits 600` (use `command copilot …` if your `copilot` alias already adds a cap); consider `--max-autopilot-continues 3` (default 5).
4. **After a long idle, start fresh** with a short written summary instead of resuming a stale session.
5. **Don't idle inside a session:** poll (`sleep`, `read_bash`) at ≤240 s; start long interviews or planning in a fresh session.
6. **Batch independent tool calls** in one message; hand noisy build/test loops to a helper agent.
7. **Route helpers to a cheaper model** with `/subagents` (explore, task).
8. **Keep the fixed prompt lean:** every MCP tool costs ≈450 tokens per call; check `/context`; review large instruction files; confirm `/model` is what you intend, since config defaults can go stale.

Example (zsh):

```
alias copilot='copilot --max-ai-credits 1000'
alias copilot-long='copilot --context long_context'
```

6. Check your own usage (local, read-only)

The CLI's local store has one row per model call (schema checked on CLI 1.0.95). In the CLI: `/usage`, `/context`, `/limits`.

```
# Top sessions by AIU
sqlite3 -readonly ~/.copilot/session-store.db "
  SELECT substr(session_id,1,8) AS session, ROUND(SUM(total_nano_aiu)/1e9,1) AS aiu, COUNT(*) AS calls
  FROM assistant_usage_events GROUP BY session_id ORDER BY aiu DESC LIMIT 10;"

# AIU per model (is helper routing working?)
sqlite3 -readonly ~/.copilot/session-store.db "
  SELECT model, ROUND(SUM(total_nano_aiu)/1e9,1) AS aiu, COUNT(*) AS calls,
         ROUND(SUM(total_nano_aiu)/1e9/COUNT(*),2) AS aiu_per_call
  FROM assistant_usage_events GROUP BY model ORDER BY aiu DESC;"
```

7. Quality guardrail and risks

- **A saving only counts if quality holds.** Baseline indicators: 3.0 mid-task user stops per 100 messages, 1.3 errors or retries per 100 assistant turns, 276 tasks marked complete. A lever is flagged when an indicator is >25% worse for two consecutive weeks.
- **A smaller window can lose detail** (compaction at ≈160k). Mitigations: hand-over notes, searchable history, `copilot-long` on request.
- **Session logs are sensitive.** Password/secret assignments appear in ≈10% of sessions (35 of 350), plus private keys, cloud keys and tokens (counts only; values were never read). The logbook masks them before storing (0 pattern matches in 2.25M strings checked) and stays local (socket-only database). Do the same if you analyse your own logs.
- **A permission hook that approves ends the CLI's own checks for that request**, so it stays tiny, read-only and in observation mode.
- **Projections, not results.** Estimates overlap; totals are lower bounds.

8. Open points and next steps

- **Open issue:** the 1,000 AIU cap is missing in about half of the new sessions (6 of 11 since 9 Oct, test runs excluded); cause unknown.
- ≈13 Oct: review the 3-day permission-hook log and decide whether to enable it (optional, payoff ≈0).
- ≈16 Oct: first before/after comparison; start cheaper helper models and the working-habit instructions.
- Then: hand-over notes for fresh sessions, then history search (only if tests support it); weekly report review.
- **Cost of the project:** ≈6.6k AIU to build (≈5% of recorded spend). Average spend is ≈12.4k AIU/week (≈19k over the five full weeks since 31 Aug), so a 30–40% saving is ≈3.7–5.0k AIU/week: payback in under two weeks if it materialises.

Discussion points for the group

1. Adopt the default context window and a spending cap as team defaults? Which cap value?
2. Sub-agent model policy: which helpers on which model?
3. The team-owned `AGENTS.md` is ≈32 KB (≈8k tokens), re-read on every call and untouched so far: worth trimming?
4. Are the quality indicators and the 25% rollback rule acceptable?

Sources and caveats

- `PLAN.md` §2–§4, §7, §10, §12 and the management report of 10 Oct 2026 (`docs/manager-report-2026-10-10.html`).
- Two data cuts: the baseline analysis (8 Jun – 9 Oct, 188 sessions with usage, ≈129.4k AIU) and the 10 Oct report (203 sessions, ≈140k AIU). Shares differ by a point or two between them.
- One workstation, one user: your patterns may differ.
- Prices are the Sonnet-5 table (AIU per 1M tokens: input 200, cache read 20, cache write 250, output 1,000). The 5-minute cache lifetime was observed for that model and may differ for others.